

SIM_LGP30 Simulator Manual

Author: Neil Higgins

Applicable to: SIM_LGP30 Version 1.0

Contents

Contents.....	1
Disclaimer.....	3
Acknowledgements.....	3
Introduction	3
Why?	3
Platform Dependency	3
Current Status	3
Future Development.....	3
Physical Drum Memory.....	3
Floating Point Interpretive System Example.....	4
Cross-Platform Version	4
Details and Related Thoughts	4
Support	4
Terminology and Conventions.....	4
Installation	5
Invocation	5
Commands	5
Command Arguments and Outputs.....	7
Simulated Machine Input.....	9
Loaders.....	9
Hex Loader (hex2drum) for Hex Files	9
Program Input Routine for Paper Tape Files.....	9
V Loader (tape2drum) for Paper Tape Files.....	10
Technical Description.....	10
Notes.....	11
My Blackjack Saga	11
References	12
Appendix 1 - Instructions in Functional Order.....	14
Appendix 2 - Instructions in Opcode Order	16
Appendix 3 - Encodings for 4-bit Bitfields.....	17

Appendix 4 – Friden Flexowriter Input and Output Encodings..... 18

Appendix 5 –Demonstration Program 19

 Source Code 19

 Assembly 19

 Execution..... 20

Appendix 6 –Blackjack Session (v loader method) 21

Appendix 7 – Blackjack Session (bootstrap method)..... 22

Disclaimer

This manual and the SIM_LGP30 simulator which it describes are free (as in beer) and free (as in thought). Do with them as you will, provided that you accept all of the consequences.

Acknowledgements

Special thanks to Jürgen Müller for his prompt and helpful responses to my questions.

Introduction

SIM_LGP30 is a simulator for the LGP-30 computer. SIM_LGP30 is written in Python.

The focus of the product is utility, not authenticity. As such, historical LGP-30 conventions are opportunistically abandoned in favour of more modern conventions.

Cycle accuracy is not an objective of this project.

Why?

Why write an LGP-30 simulator in Python when other excellent simulators already exist, e.g. the simh LGP-30 simulator [5], the LittleGP-30 [6] and George Weigt's simulator[7]?

The author was motivated by the need for a portable and hackable simulator capable of running on a microcontroller. The target language for the simulator is MicroPython [9], and the target platform is a PyBoard [10].

Platform Dependency

There is a platform dependency caused by Python's inability to natively perform non-blocking character I/O on the console. The version on GitHub was developed on Windows 11 and will not run on Linux without modification. Non-blocking character I/O on the Linux console is possible: Use Google or your favourite AI to find out how.

Current Status

- Correctly runs code produced by the companion cross assembler, XASM_LGP30
- Correctly loads v-formatted paper tape files using the v loader
- Correctly boots through to the program input routine and loads paper tape files
- Correctly runs Mel's Blackjack program (see My Blackjack Saga below)

Future Development

Physical Drum Memory

If all goes well, SIM_LGP30 will be interfaced to a physical drum memory replica (under construction but far from complete at the time of writing). This work will be the subject of another document.

Floating Point Interpretive System Example

I have a FORTRAN program which finds the real and complex roots of a polynomial. I have run this program on a PiDP-8, a PDP-11, a PiDP-11 and a PDP-10. I considered transcoding it to LGP-30 machine code, but it turns out I'm not that much of a masochist. I think I could transcode it to the LGP-30 Floating Point Interpretive System.

Cross-Platform Version

One suggestion for a cross-platform implementation is to embed the simulator in a cross-platform GUI interface. In this way, the simulator console, the LGP-30 console, debug output and error output could be separated, and non-blocking I/O used as required.

Details and Related Thoughts

For anyone interested enough to want more detail, I have collected related information (much of it removed from the first draft of this document) in a companion document, "SIM_LGP30 – Supplementary Details" (in the GitHub repository under docs).

Support

The program has been written to meet the author's personal requirements. It has been shared for what it is worth, but major changes are not envisaged. If you wish, report installation problems, bugs and feature requests using the relevant GitHub features. This is the only way you will get a response, which in any case is not guaranteed.

Terminology and Conventions

The term "sexadecimal" is used to refer to the LGP-30's unique flavour of hexadecimal (0 – 9, f, g, j, k, q, w).

"C/pc" refers to the LGP-30 C (program counter) register.

"R/ir" refers to the LGP-30 R (instruction) register.

"A/ac" refers to the LGP-30 A (accumulator) register.

In the LGP-30 documentation, memory addresses are typically expressed in decimal modulo 64 format, i.e. two decimal digits in 0 – 63 for the track number, concatenated with two decimal digits in 00 – 63 for the sector number. For simplicity, SIM_LGP30 does **not** support this format – address literals occupy the contiguous range of 0 – 4095.

Installation

The author has minimal packaging and installation skills. The following should work:

1. Clone the GitHub repository. This will create a structure which looks like:

Structure	Actual
project_dir	sim_lgp30
docs_dir (documentation)	docs
src_dir	src
modules_dir (python files)	sim_lgp30
tests_dir (tests)	tests
venv_dir (interpreter, libraries)	venv
etc.	etc.

2. In the project directory, type:

```
python -m venv venv (create a virtual environment)
.\venv\scripts\activate (activate the virtual environment)
pip install -r requirements.txt (install the dependencies)
pip install -e . (the dot is a literal - current directory)
```

3. In the project directory, type:

```
pip install pyinstaller
pyinstaller --onefile .\src\sim_lgp30\simulator.py (this will generate a .exe file in the dist directory)
```

You should be able to run the .exe file directly. If you want, you can copy the .exe file to a Windows Program Files subdirectory and create shortcut to a command window which has the .exe file on its path.

Invocation

In the command window, type:

```
simulator <path_to_hex_file> [--ac=<inital_accumulator_value>] [--st=<start_address>] [-r (run)]
```

The format for the --ac and --st options must be adhered to exactly - it is a requirement of Python's argparse library.

Commands

Minimal command abbreviations are accepted.

accumulator [value]	Set/show A/ac value
atqdeposit address value qvalue	Set the memory location at address to value, encoded as a fixed point quantity at qvalue
atqexamine address qvalue	List the memory contents at address, assuming it is a fixed point quantity at qvalue

blackjack [on off]	Set/show blackjack mode (on / off). When blackjack mode is on, SIM_LGP30 ignores z instructions
cd path	Change the working directory to path
colour [black red]	Set/show the current printer colour. The initial colour is DEFAULT_COLOUR in friden_constants.py.
convert TTSS	Convert TTSS to decimal. There should be no space between TT (track) and SS (sector)
deposit address value	Set the memory location at address to value
directory	List files in the current working directory
drum2hex path_to_hex_file	Transcribe the drum and C/pc to an Intel Hex file. Before doing so, check that C/pc is set to the program start address
eatqexamine address qvalue (extended atqexamine)	List the memory contents at address (the result of an m multiply), and the contents of A/ac (the result of an n multiply), assuming that, appropriately concatenated, they form a fixed-point quantity at qvalue
examine range_start [range_end]	List memory contents from address range_start [to address range_end]
exit	Terminate the simulator
hex2drum path_to_hex_file	Ingest an Intel hex file onto the drum and C/pc
input [kb tape]	Set/show the input source – keyboard (kb) or paper tape file (tape)
memreset	Empty memory and re-initialise registers to Python's None
mount [path_to_tape_file 0]	Set/show the tape file to be ingested by an LGP-30 program using i instructions. 0 closes an open file
pwd	Show the current working directory
reset	Resets C/pc to the most recent value set by the start address in the hex file on the command line, the --st option on the command line, a hex2drum command, a tape2drum command or a startaddr command
run	Step continuously, suppressing the simulator dialogue, until a non-Friden character is received from the console, or the program terminates
startaddr [address]	Set/show C/pc value
step	Single step. The first step command after C/pc is set or reset fetches an instruction and displays the simulator state. The next and subsequent step commands execute the pending instruction, display changes, then fetch the next instruction and display the simulator state, pending execution
switches [list of up to 4 switches in 32, 16, 8, 4 0]	Set the switches (0 clears all). Any or all switches can be set. The switches are initially cleared

tabs [list of up to eight tab stops 0]	Set tab stops on the simulated Friden Flexowriter (0 clears all). There are no tab stops initially
tape2drum path_to_tape_file	Ingest a v loader-formatted tape file onto the drum (and C/pc if a transfer record is found). This is an alternative to booting through to a tape loader in order to load a tape
tapemode [4 6]	Set/show the tape reader mode (4 or 6 bits). The initial mode is FRIDEN_DEFAULT_MODE in friden_constants.py
tcs [on off]	Set/show the state of the transfer control switch (TCS). At this stage, the TCS cannot be toggled “on the fly”
trace num_cycles	Like step , except that multiple instruction cycles, including the initial pending instruction cycle, are executed (up to num_cycles or a machine halt). trace 1 is the same as step

Command Arguments and Outputs

Given the choice of alignment of the operand address within the LGP-30 instruction word, there is no “natural” or easy” mental model for decoding raw binary. The only slight relief is that opcodes align cleanly with a sex or hex byte. For the same reason, constants used for address “indexing” are hard to read. The so-called “decimal” form of address encoding (dual decimal modulo 64) used in LGP-30 documentation exacerbates this problem. For this reason, quantities in the simulator dialogue are decoded and displayed in multiple ways, making the output quite dense. The following table describes the format for each command. Leading zeros are suppressed for decimal values.

Input values to the **accumulator** and **deposit** commands should be **32-bit** values. Displayed values are **32-bit** values. It is valid to deposit a 32 bit value in the accumulator. A warning is given if a value with a non-zero LSB is stored to or retrieved from a memory location.

Command	Output Fields
accumulator	In: v (optional) v: value - decimal Out: A/ac: v ssssssss hhhhhhhh o m oa ot os v: value - decimal ssssssss: value - sexadecimal hhhhhhh: value - hexadecimal o: opcode - decimal m: mnemonic oa: operand address - decimal ot: operand address (track) - decimal os: operand address (sector) - decimal

atqdeposit	In: a v q a: address - decimal v: value - float q: q value - decimal Out: a: v q a: address - decimal v: value - float q: q value - decimal
atqexamine	In: a q a: address a - decimal q: q value - decimal Out: a: v q a: address - decimal v: value - float q: q value - decimal
eatqexamine	In: a q a: address - decimal q: q value - decimal Out: a: v q a: address - decimal v: value - float q: q value - decimal
deposit	In: a v a: address - decimal v: value - decimal Out: a t s: v ssssssss hhhhhhhh o m oa ot os a: address - decimal t: track - decimal s: sector - decimal v: value - decimal sssssss: value - sexadecimal hhhhhhh: value - hexadecimal o: opcode - decimal m: mnemonic oa: operand address - decimal ot: operand address (track) - decimal os: operand address (sector) - decimal
examine	In: a a: address - decimal Out: a t s: v ssssssss hhhhhhhh o m oa ot os a: address - decimal t: track - decimal s: sector - decimal v: value - decimal sssssss: value - sexadecimal hhhhhhh: value - hexadecimal o: opcode - decimal m: mnemonic oa: operand address - decimal ot: operand address (track) - decimal os: operand address (sector) - decimal

step	Out: C/pc: a t s R/ir: v ssssssss hhhhhhhh o m oa ot os (disassembled R/ir) Oprd: v ssssssss hhhhhhhh o m oa ot os (disassembled operand) A/ac: v ssssssss hhhhhhhh o m oa ot os (disassembled A/ac) BrPt: breakpoint switches (sum of values – 32, 16, 8, 4) TrCS: transfer control switch If the operand changed: Previous disassembled operand --> Current disassembled operand
trace	As for step, per instruction

Simulated Machine Input

Because of the arcane manner in which the LGP-30 takes input, input must be terminated with a stop-reader code, which is a prime (') character (not a carriage return). When an i instruction is blocked waiting for input, control can be returned to the simulator console by typing a non-Friden character (Escape works well).

SIM_LGP30 has a keyboard timeout, set to FRIDEN_TIMEOUT (seconds) in friden_constants.py. On expiry of the timeout, control returns to the simulator console.

The Friden emulator does not implement “local echo” – so unless a running program echoes keyboard input, nothing is seen when input is typed. I am not sure if this is authentic or not. When reading from tape, the emulator outputs a dot followed by a newline for each newline read, to indicate that the program is working.

Loaders

Hex Loader (hex2drum) for Hex Files

Drum images may be ingested from and exported to files in Intel Hex format [4]. This is the output format of the companion cross assembler, XASM_LGP30.

Intel Hex format is a time honoured, well documented, robust and easy-to-use format for application binaries. Refer to the references for complete information.

The format is 32 bits (8 **hex**adecimal bytes), big-endian, **LSB** set to zero. Addresses are **linear**, i.e. address 0xff is literally address 255 (track 3, sector 63).

Multiple hex files can be ingested in order to compile an application and the required libraries. The compiled application can then be exported as a composite drum image.

Program Input Routine for Paper Tape Files

The program input routine is an elementary assembler and relocating loader. It is **not implemented** in the current version of SIM_LGP30. The program input routine can be loaded via the bootstrap process described in the LGP-30 documentation. Paper tape formatting rules are detailed in the LGP-30 Programming Class Notes [2].

V Loader (tape2drum) for Paper Tape Files

The v loader is a subset of the program input routine. It can be used for loading assembled programs such as Blackjack.

The v loader:

- Accepts descriptive records type of '\$' followed by descriptive text and terminated by a stop character. This is unauthentic and was implemented for testing purposes only
- Otherwise accepts only "v blocks" (sexadecimal records prefixed by a "v record") and "transfer records" (indicating the program start address). Tape heading and boot loader sections of most paper tape files should be removed
- Accepts both 'l' (lower case L) and '1' (numeric one) as representations of the value 1. This is also unauthentic
- Checks that the record count digits in a transfer record are valid sex digits, but otherwise ignores them
- In a v block, accepts but ignores any number of records after the sexadecimal records and before the next v record
- The form of the address in a transfer record is has not been verified. At present, the v loader accepts sexadecimal, however the documentation specifies "decimal" (presumably TTSS), which is inconsistent with v records.

Technical Description

In SIM_LGP30, registers are initialised to Python's **None**. Memory locations are stored in a Python dictionary and do not exist until initialised; uninitialized locations return **None**. This is motivated by the simple desire to catch bad programming practice; however it is both unauthentic and unphysical.

It is also a two-edged sword, because it introduces extra halting conditions: As one would expect, the simulator halts on z instructions (appropriately qualified by the Break Point switches) and on arithmetic overflows. **It also halts when the simulator fetches an instruction from uninitialised memory, and when it is ordered to operate on uninitialised data.**

This latter is potentially fatal to some (perhaps many) programs, including Blackjack, which use the c instruction to initialise A/ac. There are two irreconcilable use cases for the c instruction – one to set A/ac to zero without knowing or caring about its value, the other to save a previously computed value before setting A/ac to zero.

It is also potentially fatal to some (perhaps many) programs which execute an i instruction before initialising A/ac.

Faithful execution can be achieved by setting LGP30_STRICT in simulator_constants.py to False. Setting this constant to True will result in the simulator issuing a warning for the c and i instructions, and halting for other cases of a non-initialised operand.

After initialisation, C/pc may be set from the start address in the hex file on the command line, the --st option on the command line, a **hex2drum** command, a **tape2drum** command or a **startaddr** command. A/ac may be set from the --ac option on the command line or the **accumulator** console command. Memory locations may be set from a hex file, a tape file, the **deposit** command or the **atqdeposit** command.

Registers C/pc, R/ir and A/ac are shadowed by objects which retain both the “current” and the “previous” value. A single Memory object shadows the most recently accessed memory location. This enables the effects of instruction execution to be traced.

Instruction execution (retrieving operands, calculating results and storing results) is atomic for tracing purposes. Extra reads and writes are used when processing simulator commands.

If ANSI_TERMINAL in friden_constants.py is set to True:

- The console window colour scheme is initialised to black-on-white using an ANSI escape sequence. An LGP-30 program may subsequently toggle the foreground colour between black and red – both colours will be visible.
- SIM_LGP30 outputs ANSI escape sequences for black and red, toggling between them when the Friden colour shift code is printed. Otherwise it outputs “<blk>” or “<red>” as appropriate.

Notes

1. Overall impressions of the LGP-30 vary. While the designer, Stanly Frankel, is to be commended for reducing the hardware cost by replacing expensive, unreliable valves with more compact and inexpensive diodes, the resulting instruction set architecture is (even by the standards of the day) both slow and inelegant.
2. In discussions pursuant to the SNOCOM paper [11], the following comment is made: *“Despite a policy which encourages the use of diagnostic routines and obviates the need for programme checking on a “step-by-step” basis, almost half the useful time has been spent on programme checking. [This] ... reflects the influence of the relatively large number of inexperienced programmers ...”* The foregoing would have been written just after the word “bug” entered the computing lexicon. “Programme checking” in this context undoubtedly means “debugging” – evidence (if we need any) that programming the LGP-30 was laborious and error-prone.
3. Dare I say: The key to understanding the LGP-30 is the realisation that it is “a fixed-point calculator with frills.” Forget about addressing modes, forget about integer arithmetic, forget about text processing. Understand “the meaning of q” and think about multiplication, division, addition and subtraction.
4. The author has not found any examples of hardware verification programs which would be useful for testing the simulator. The only easily repeatable test seems to be, “Does it run Mel’s Blackjack program?”
5. The existence of a “Floating Point Interpretive System” [12] for the LGP-30 is a clear indication that, even in those days, users were not fussed on keeping track of exponents manually. They were more than happy to trade off precision and speed in order to relieve themselves of this task. To this day, 32-bit (“single precision”) floating point numbers are still regarded as acceptable for many purposes.

My Blackjack Saga

Most people have a Blackjack war story. Here’s mine.

I could load Blackjack using the v loader, but it would not run. It kept halting, even after forced resumptions, over and over and over. I hit upon a website that contained what turned out to be an incorrect disassembly of the program. After discovering that this disassembly was incorrect, I prepared my own disassembly and started hand tracing (all this took time, as I’m sure you will

understand). I discovered to my surprise that the program appears to contain z (stop) instructions in many places. Why would the program run at all?

I assumed that some of the supposed z instructions would turn out to be constants that happen to have zero in the opcode field. Even so, I kept hunting for prior experience. ChatGPT contributed comments and analyses that were logical, but not especially helpful – I had already reached the same conclusions.

I stumbled across George Weigt's simulator called Mel [7], which is written in C and runs Blackjack perfectly. The real surprise was that George's program ignores z instructions! Lo and behold, when I patched SIM_LGP30 to make it ignore z instructions, it also ran Blackjack perfectly.

About then I consulted (not for the first time) Jürgen Müller of LittleGP-30 fame. Jürgen confirmed that Blackjack hits a z instruction early in the piece for no obvious reason. I added a "Blackjack Mode" to SIM_LGP30 – in Blackjack mode it ignores z instructions – well and good, but unsatisfying. Shouldn't Blackjack "just work"?

Along the way, I got to thinking about the t instruction. A t instruction with the MSB set (I call it *t) will branch unconditionally if the transfer control switch (TCS) is toggled on. This feature is used to enable tracing in the Floating Point Interpretive System. Could it be that each z instruction is prefixed by *t instruction, and blackjack should be run with TCS on? The disassembly didn't really support this hypothesis but – worth a try. I implemented the TCS switch and it made no difference.

I downloaded the simh LGP-30 simulator, then loaded and ran Blackjack. simh behaves differently but still not entirely as expected. It stops after each game, and must be restarted once within each game.

So: The operation of the Blackjack program is an enduring mystery. Perhaps on a real LGP-30 there is a combination of front-panel switches that allows Blackjack to run uninterrupted.

If anyone chooses to read this account and knows the answer, please enlighten me.

References

The following references were all "alive" at the time of writing.

1. LGP-30 Programming Manual – https://www.bitsavers.org/pdf/royalPrecision/LGP-30/LGP-30_Programming_Manual_Apr57.pdf
2. LGP-30 Programming Class Notes – http://computermuseum.informatik.uni-stuttgart.de/ftp/lgp30/docs/LGP30_ProgrammingClassNotes.pdf
3. LGP-30 Subroutine Manual – https://www.bitsavers.org/pdf/royalPrecision/LGP-30/LGP-30_Subroutine_Manual_Oct60.pdf
4. Intel Hex Format - https://en.wikipedia.org/wiki/Intel_HEX
5. simh LGP-30 simulator Manual - https://simh.trailing-edge.com/pdf/lgp_doc.pdf
6. LittleGP-30 - <https://www.e-basteln.de/computing/lgp30/lgp30/>
7. GitHub, George Weigt, mel - <https://github.com/georgeweigt/mel>. An LGP-30 simulator written in C, expressly to run the Blackjack program
8. University of Stuttgart Computer Museum, LGP-30 artefacts - <http://computermuseum.informatik.uni-stuttgart.de/ftp/lgp30/>
9. MicroPython - <https://micropython.org/>
10. PyBoard - <https://store.micropython.org/product/PYBv1.1>

11. Wong D, "The Logical Design of the General Purpose Digital Computer SNOCOM," The Institution of Engineers Australia Journal, June 1962. SNOCOM was a transistorised replica of the LGP-30, which saw service with Australia's Snowy Mountains Authority
12. LGP-30 Floating Point Interpretive System (24.2) Programming Notes,
[https://upload.wikimedia.org/wikipedia/commons/0/07/LGP-30 Floating Point Interpretive System Programming Notes Nov59.pdf](https://upload.wikimedia.org/wikipedia/commons/0/07/LGP-30_Floating_Point_Interpretive_System_Programming_Notes_Nov59.pdf)

Appendix 1 - Instructions in Functional Order

The following table is a close replica of the original documentation, with a few changes made for clarity.

Mnemonic	Binary	Example	Description
b	0001	b 2000	Bring: Replace the word in the A register with the word in memory location 2000. The contents of memory location 2000 are unaffected.
h	1100	h 2000	Store and hold: Replace the word in memory location 2000 with word in the A register. The contents of the A register are unaffected.
c	1101	c 2000	Store and clear: Replace the word in memory location 2000 with the word in the A register, then zero the A register.
a	1110	a 2000	Add: Add the word in memory location 2000 to the A register and place the result in the A register. The contents of memory location 2000 are unaffected.
s	1111	s 2000	Subtract: Subtract the word in memory location 2000 from the A register and place the result in the A register. The contents of memory location 2000 are unaffected.
m	0111	m 2000	Multiply upper: Multiply the number in the A register by the number in memory location 2000 and place the sign and the most significant thirty bits of the product in the A register. The contents of memory location 2000 are unaffected.
n	0110	n 2000	Multiply lower: Multiply the number in the A register by the number in memory location 2000 and place the least significant thirty-one magnitude bits of the product in the sign bit and thirty magnitude bits of the A register. The contents of memory location 2000 are unaffected.
d	0101	d 2000	Divide: Divide the number in the A register by the number in memory location 2000 and place the quotient, rounded to thirty bits, in the A register. The contents of memory location 2000 are unaffected.
e	1001	e 2000	Extract: (Logical AND) Place zeroes in the word in the A register wherever there are zeroes in location 2000 but otherwise leave the word in the A register unchanged. The contents of memory location 2000 are unaffected.
y	0010	y 2000	Store address: Replace the address portion of the word in memory location 2000 with the address portion of the word in the A register. The contents of the A register are unaffected. The LGP-30 has no index register. This instruction can be used to self-modify code to index an array.
r	0011	r 2000	Return address: Add one to the (post-fetch) value of the C register (do not update the C register), then replace the address portion of the word in location 2000 with this value . R is followed immediately by an unconditional transfer instruction to the beginning of the subroutine. This instruction is used to patch an unconditional transfer instruction at the end of a subroutine, redirecting execution to the instruction following the invoking transfer. R can also be used to patch a Test instruction in order to create a conditional return.
u	1010	u 2000	Unconditional transfer: Replace the word in the C register with the address portion of this instruction.

t	1011	t 2000	Test: (Conditional transfer) If the sign bit of the A register is a 1, the test instruction has the effect of an unconditional transfer. If the sign bit of the A register is a 0, the next instruction in the normal sequence is executed. But note: When the transfer control switch is on, if a test instruction has the sign bit (normally unused in an instruction word) set, a transfer will take place unconditionally, i.e. regardless of the accumulator's value.
z	0000	z 0000	Stop: Stop computation. If computation is to stop, the address portion of the stop instruction is usually of no significance. However, the stop instruction has a special characteristic: Four break point switches (called Break Point 32, Break Point 16, Break Point 8 and Break Point 4) are supported by the z instruction. There is a 1:1 correspondence between these switches and the four most significant bits of the track number. Computation does not stop if a break point switch is latched down and the corresponding bit is set in the track number.
p	1000	p 2000	Print: Execute the typewriter keyboard function indicated by the 6 track bits. The print order has no effect on the contents of any memory location, the A register, or the C register. For example, p 2000 has 010100 in the track bits which is the code for a back space on the typewriter. The execution of p 2000 results in the typewriter back spacing.
i	0100	i 0000	Input: The address portion of this instruction is always 0000. It is always preceded by the instruction p 0000. After a p 0000 instruction starts the tape reader, an i 0000 instruction transfers into the last 4 bit positions of the A register the first 4 bits of the typewriter code for the first character read on the tape. When the second character is read, the bits representing the first character are shifted left into the next to last four bit positions of the A register and the first four bits of the typewriter code of the second character on tape are placed in the last four bit positions of the A register. This process continues up to eight times to fill the entire A register until a stop code (100000) appears on the tape. The stop code stops the tape reader and sends a start signal to the computer so that the instruction following i 0000 in memory is executed. Often this next instruction is a h or c instruction so that the characters read into the A register can be stored in a memory location.

Appendix 2 - Instructions in Opcode Order

Binary	Decimal	Hex	Sex	Mnemonic	Instruction
0000	0	0	0	z	Stop
0001	1	1	1	b	Bring
0010	2	2	2	y	Store address
0011	3	3	3	r	Return address
0100	4	4	4	i	Input
0101	5	5	5	d	Divide
0110	6	6	6	n	Multiply lower
0111	7	7	7	m	Multiply upper
1000	8	8	8	p	Print
1001	9	9	9	e	Extract
1010	10	a	f	u	Unconditional transfer
1011	11	b	g	t	Test
1100	12	c	j	h	Store and hold
1101	13	d	k	c	Store and clear
1110	14	e	q	a	Add
1111	15	f	w	s	Subtract

Appendix 3 - Encodings for 4-bit Bitfields

Bitfield	Decimal	Sexadecimal	Hexadecimal
0000	0	0s0	0x0
0001	1	0s1	0x1
0010	2	0s2	0x2
0011	3	0s3	0x3
0100	4	0s4	0x4
0101	5	0s5	0x5
0110	6	0s6	0x6
0111	7	0s7	0x7
1000	8	0s8	0x8
1001	9	0s9	0x9
1010	10	0sf	0xa
1011	11	0sg	0xb
1100	12	0sj	0xc
1101	13	0sk	0xd
1110	14	0sq	0xe
1111	15	0sw	0xf

Appendix 4 – Friden Flexowriter Input and Output Encodings

Code Order

Alphanumeric Keys		Output Code		Input Code
Upper Case	Lower Case	Decimal (track)	6-bit (track)	4-bit (6-bit) binary
Start reader [†]		0	000000	
Z	z	1	000001	0000 (01)
)	0	2	000010	0000 (10)
Space		3	000011	0000 (11)
Lower case [†]		4	000100	
B	b	5	000101	0001 (01)
L	1	6	000110	0001 (10)
_	-	7	000111	0001 (11)
Upper case [†]		8	001000	
Y	y	9	001001	0010 (01)
*	2	10	001010	0010 (10)
=	+	11	001011	0010 (11)
Colour shift [†]		12	001100	
R	r	13	001101	0011 (01)
"	3	14	001110	0011 (10)
:	;	15	001111	0011 (11)
Carriage return [†]		16	010000	
I	i	17	010001	0100 (01)
Δ (delta)	4	18	010010	0100 (10)
?	/	19	010011	0100 (11)
Backspace [†]		20	010100	
D	d	21	010101	0101 (01)
%	5	22	010110	0101 (10)
]	.	23	010111	0101 (11)
Tab		24	011000	0000 (00)
N	n	25	011001	0110 (01)
\$ (dollar)	6	26	011010	0110 (10)
[	,	27	011011	0110 (11)
M	m	29	011101	0111 (01)
Π (pi)	7	30	011110	0111 (10)
V	v	31	011111	0111 (11)
In: Stop reader Out: ' (prime)		32	100000	
P	p	33	100001	1000 (01)
Σ (sigma)	8	34	100010	1000 (10)
O	o	35	100011	1000 (11)
E	e	37	100101	1001 (01)
(	9	38	100110	1001 (10)
X	x	39	100111	1001 (11)
U	u	41	101001	1010 (01)
F	f	42	101010	1010 (10)
T	t	45	101101	1011 (01)
G	g	46	101110	1011 (10)
H	h	49	110001	1100 (01)
J	j	50	110010	1100 (10)
C	c	53	110101	1101 (01)
K	k	54	110110	1101 (10)
A	a	57	111001	1110 (01)
Q	q	58	111010	1110 (10)
S	s	61	111101	1111 (01)
W	w	62	111110	1111 (10)
Delete**		63	111111	

[†] Not used in input

* In this context, "Delete" refers to a paper tape delete code – all columns punched – meaning that the character will be ignored

Alpha Order

Alphanumeric Keys		Output Code		Input Code
Upper Case	Lower Case	Decimal (track)	6-bit (track)	4-bit (6-bit) binary
A	a	57	111001	1110 (01)
B	b	5	000101	0001 (01)
C	c	53	110101	1101 (01)
D	d	21	010101	0101 (01)
E	e	37	100101	1001 (01)
F	f	42	101010	1010 (10)
G	g	46	101110	1011 (10)
H	h	49	110001	1100 (01)
I	i	17	010001	0100 (01)
J	j	50	110010	1100 (10)
K	k	54	110110	1101 (10)
M	m	29	011101	0111 (01)
N	n	25	011001	0110 (01)
O	o	35	100011	1000 (11)
P	p	33	100001	1000 (01)
Q	q	58	111010	1110 (10)
R	r	13	001101	0011 (01)
S	s	61	111101	1111 (01)
T	t	45	101101	1011 (01)
U	u	41	101001	1010 (01)
V	v	31	011111	0111 (11)
W	w	62	111110	1111 (10)
X	x	39	100111	1001 (11)
Y	y	9	001001	0010 (01)
Z	z	1	000001	0000 (01)
)	0	2	000010	0000 (10)
L	1	6	000110	0001 (10)
*	2	10	001010	0010 (10)
"	3	14	001110	0011 (10)
Δ (delta)	4	18	010010	0100 (10)
%	5	22	010110	0101 (10)
\$ (dollar)	6	26	011010	0110 (10)
Π (pi)	7	30	011110	0111 (10)
Σ (sigma)	8	34	100010	1000 (10)
(	9	38	100110	1001 (10)
_	-	7	000111	0001 (11)
=	+	11	001011	0010 (11)
:	;	15	001111	0011 (11)
?	/	19	010011	0100 (11)
]	.	23	010111	0101 (11)
[	,	27	011011	0110 (11)
Start reader [†]		0	000000	
Space		3	000011	0000 (11)
Lower case [†]		4	000100	
Upper case [†]		8	001000	
Colour shift [†]		12	001100	
Carriage return [†]		16	010000	
Backspace [†]		20	010100	
Tab		24	011000	0000 (00)
In: Stop reader Out: ' (prime)		32	100000	
Delete**		63	111111	

[†] Not used in input

* In this context, "Delete" refers to a paper tape delete code – all columns punched – meaning that the character will be ignored

Appendix 5 –Demonstration Program

Source Code

```
; prog1.asm
; Programming Class Notes, Section V, Simple Problem
;
; Compute K * (F + G - J) / W
; F in 1500 at q = 7 23 -17.5
; G in 1501 at q = 7 6.375 -6.375
; J in 1502 at q = 7 15.8 -15.8
; K in 1503 at q = 7 3.142 6.284
; W in 1504 at q = 7 2.718 -10
; Result 15.6927 5.0743
;
.locn 1000
;
start:
    b F          ; F at q = 7
    a G          ; F + G at q = 7
    h temp
    s J          ; F + G - J at q = 7
    h temp
    m K          ; (F + G - J) * K
    h temp
    d W          ; (F + G - J) * K / W
    h result
    z 0
;
.locn 1500
;
F: .word 0      ; use atqdeposit
G: .word 0      ; use atqdeposit
J: .word 0      ; use atqdeposit
K: .word 0      ; use atqdeposit
W: .word 0      ; use atqdeposit
;
result: .word 0 ; use atqexamine
temp:   .word 0 ; debugging only
;
.end start
```

Assembly

Using the companion program XASM_LGP30.

```
> assembler prog1.asm -o prog1.hex -l prog1.lst
XASM_LGP30 Version 1.0
Listing file: prog1.lst
Source file: prog1.asm
End of pass 1 - 0 errors
End of pass 2 - 0 errors
Object file: prog1.hex
>
```

Execution

```
> simulator prog1.hex
sim_lgp30 version 0.1
Reading hex file: prog1.hex
Address: 1000 <-- 71536
Address: 1001 <-- 923508
Address: 1002 <-- 792456
Address: 1003 <-- 989048
Address: 1004 <-- 792456
Address: 1005 <-- 464764
Address: 1006 <-- 792456
Address: 1007 <-- 333696
Address: 1008 <-- 792452
Address: 1009 <-- 0
Address: 1500 <-- 0
Address: 1501 <-- 0
Address: 1502 <-- 0
Address: 1503 <-- 0
Address: 1504 <-- 0
Address: 1505 <-- 0
Address: 1506 <-- 0
Start address in hex file: 1000
--> atqdeposit 1500 23 7
1500: 23.0 7
--> atqdeposit 1501 6.375 7
1501: 6.375 7
--> atqdeposit 1502 15.8 7
1502: 15.799999952316284 7
--> atqdeposit 1503 3.142 7
1503: 3.1419999599456787 7
--> atqdeposit 1504 2.718 7
1504: 2.7179999351501465 7
--> run
Expecting keyboard input
--> atqexamine 1505 7
1505: 15.692659974098206 7
--> exit
>
```

Appendix 6 –Blackjack Session (v loader method)

```
> simulator primary_boot.hex
[ed. but note that the primary boot loader is not used in this example]
sim_lgp30 version 0.1
Reading hex file: primary_boot.hex
Address: 4032 <-- 524288
Address: 4033 <-- 262144
Address: 4034 <-- 868116
Address: 4035 <-- 524288
Address: 4036 <-- 262144
Start address in hex file: 4032
--> tape2drum bkjck.txt
Reading tape file: bkjck.txt
Address: 2880 <-- 864140
... (etc.)
... (etc.)
... (etc.)
Address: 3519 <-- 0
--> sta 2880
Start address: 2880
--> tabs 15 30 45
Tab stops: 15 30 45
--> blackjack on
Blackjack mode: On
--> run
Expecting keyboard input
None A/ac in c instruction at 2880 - substituting zero

          5 - c          7 - s
          4 - s
Card?      9 - s
Card?
          total - 18    Q - h    total - 17    score = $1.00

          J - h          K - h
          2 - s
Card?      J - c          10 - h
          Bust          total - 20    score = $0.00

          4 - d          7 - c
          Q - s
Card?
(user typed <Escape>)
Input terminated by non-Friden character
--> exit
>
```

Appendix 7 – Blackjack Session (bootstrap method)

```
> simulator primary_boot.hex
SIM_LGP30 version 1.0
Reading hex file: primary_boot.hex
Address: 4032 <-- 524288
Address: 4033 <-- 262144
Address: 4034 <-- 868116
Address: 4035 <-- 524288
Address: 4036 <-- 262144
Start address in hex file: 4032
--> input tape
Input source: tape
--> tapemode 4
Tape mode: 4
--> mount secondary_boot.txt
Mounted: secondary_boot.txt, position = 0
--> run
None A/ac in i instruction at 4033, substituting zero
.
.
(etc.)
.
.
--> sta 0
Start address: 0
--> mount bkjck.txt
Mounted: bkjck.txt, position = 0
--> run
.
.
(etc.)
.
.
Input terminated - end of tape file
--> input kb
Input source: kb
--> tabs 15 30 45
Tab stops: 15 30 45
--> blackjack on
Blackjack mode: On
--> sta 2880
Start address: 2880
--> run
Expecting keyboard input
None A/ac in c instruction at 2880 - substituting zero

          5 - c          7 - s
          4 - s
Card?      9 - s
Card?      Q - h
          total - 18    total - 17    score = $1.00

          J - h          K - h
          2 - s
Card?      J - c          10 - h
          Bust          total - 20    score = $0.00

          4 - d          7 - c
          Q - s
Card?
(user typed <Escape>)
Input terminated by non-Friden character
--> exit
>
```